# Task 1

**Create a database table and write SQL queries that:**
**Creates the employees table and inserts the employee records.**
**Displays all records using SELECT *.**
**Displays only selected columns.**
**Uses column aliases to rename the output columns.**

**Source Code:**

```
CREATE TABLE employees (
 employee_id INT PRIMARY KEY,
 employee_name VARCHAR(80) NOT NULL,
 department VARCHAR(40) NOT NULL,
 salary NUMERIC(10,2) NOT NULL,
 city VARCHAR(40),
 joining_date DATE
);

INSERT INTO employees VALUES
(101,'Aarav Sharma','Sales',45000,'Delhi','2024-01-15'),
(102,'Priya Verma','HR',38000,'Agra','2023-08-10'),
(103,'Rahul Singh','IT',62000,'Mathura','2022-05-20'),
(104,'Neha Gupta','Sales',52000,'Agra','2024-03-12'),
(105,'Kabir Khan','IT',70000,'Delhi','2021-11-01'),
(106,'Isha Patel','Finance',48000,'Jaipur','2023-02-18'),
(107,'Rohan Das','Sales',41000,'Mathura','2022-09-05'),
(108,'Meera Joshi','Finance',56000,'Delhi','2024-06-25');

-- TASK 1: SELECT
SELECT * FROM employees;

SELECT employee_name, department, salary FROM employees;

SELECT employee_name AS "Employee", salary AS "Monthly Salary" FROM employees;
```

**Output:**

```
 employee_id | employee_name | department |  salary  |  city   | joining_date
-------------+---------------+------------+----------+---------+--------------
         101 | Aarav Sharma  | Sales      | 45000.00 | Delhi   | 2024-01-15
         102 | Priya Verma   | HR         | 38000.00 | Agra    | 2023-08-10
         103 | Rahul Singh   | IT         | 62000.00 | Mathura | 2022-05-20
         104 | Neha Gupta    | Sales      | 52000.00 | Agra    | 2024-03-12
         105 | Kabir Khan    | IT         | 70000.00 | Delhi   | 2021-11-01
         106 | Isha Patel    | Finance    | 48000.00 | Jaipur  | 2023-02-18
         107 | Rohan Das     | Sales      | 41000.00 | Mathura | 2022-09-05
         108 | Meera Joshi   | Finance    | 56000.00 | Delhi   | 2024-06-25
(8 rows)
```

```
 employee_name | department |  salary
---------------+------------+----------
 Aarav Sharma  | Sales      | 45000.00
 Priya Verma   | HR         | 38000.00
 Rahul Singh   | IT         | 62000.00
 Neha Gupta    | Sales      | 52000.00
 Kabir Khan    | IT         | 70000.00
 Isha Patel    | Finance    | 48000.00
 Rohan Das     | Sales      | 41000.00
 Meera Joshi   | Finance    | 56000.00
(8 rows)
```

```
   Employee   | Monthly Salary
--------------+----------------
 Aarav Sharma |       45000.00
 Priya Verma  |       38000.00
 Rahul Singh  |       62000.00
 Neha Gupta   |       52000.00
 Kabir Khan   |       70000.00
 Isha Patel   |       48000.00
 Rohan Das    |       41000.00
 Meera Joshi  |       56000.00
(8 rows)
```

## Explanation

**First, I created the employees table with six columns and inserted eight employee records into it. Then, I used SELECT * to display all columns and rows of the table. After that, I selected only the employee_name, department, and salary columns to display the required data. Finally, I used the AS keyword to give the columns the new names "Employee" and "Monthly Salary" in the output. This task helped me understand how to create a table, insert data, and retrieve data using the SELECT statement.**

# Task 2

**Write SQL queries that:**
**Filter employees whose salary is greater than 50000 using WHERE.**
**Combine two conditions using AND (city is Delhi and salary is at least 50000).**
**Sort the employees by salary in descending order using ORDER BY.**
**Use aggregate functions COUNT, SUM, AVG, MIN and MAX.**

**Source Code:**

```sql
SELECT * FROM employees WHERE salary > 50000;

SELECT employee_name, salary
FROM employees
WHERE city='Delhi'
AND salary >= 50000;

SELECT employee_name, salary
FROM employees
ORDER BY salary DESC;

SELECT COUNT(*) AS employee_count,
SUM(salary) AS total_salary,
ROUND(AVG(salary),2) AS average_salary,
MIN(salary) AS minimum_salary,
MAX(salary) AS maximum_salary FROM employees;
```

**Output:**

```
 employee_id | employee_name | department |  salary  |  city   | joining_date
-------------+---------------+------------+----------+---------+--------------
         103 | Rahul Singh   | IT         | 62000.00 | Mathura | 2022-05-20
         104 | Neha Gupta    | Sales      | 52000.00 | Agra    | 2024-03-12
         105 | Kabir Khan    | IT         | 70000.00 | Delhi   | 2021-11-01
         108 | Meera Joshi   | Finance    | 56000.00 | Delhi   | 2024-06-25
(4 rows)
```

```
 employee_name |  salary
---------------+----------
 Kabir Khan    | 70000.00
 Meera Joshi   | 56000.00
(2 rows)
```

```
 employee_name |  salary
---------------+----------
 Kabir Khan    | 70000.00
 Rahul Singh   | 62000.00
 Meera Joshi   | 56000.00
 Neha Gupta    | 52000.00
 Isha Patel    | 48000.00
 Aarav Sharma  | 45000.00
 Rohan Das     | 41000.00
 Priya Verma   | 38000.00
(8 rows)
```

```
 employee_count | total_salary | average_salary | minimum_salary | maximum_salary
----------------+--------------+----------------+----------------+----------------
              8 |    412000.00 |       51500.00 |       38000.00 |       70000.00
(1 row)
```

## Explanation

**First, I used the WHERE clause to display only those employees whose salary is greater than 50000. Then, I used AND to apply two conditions together, so only employees living in Delhi with a salary of 50000 or more were displayed. After that, I used ORDER BY salary DESC to arrange the employees from the highest salary to the lowest salary. Finally, I used the aggregate functions COUNT(), SUM(), AVG(), MIN() and MAX() to find the number of employees, total salary, average salary, minimum salary and maximum salary in a single query.**

# Task 3

**Write an SQL query that:**
**Groups the employees by department using GROUP BY.**
**Displays the number of employees and the average salary of each department.**
**Shows only those departments that have at least 2 employees using HAVING.**
**Sorts the result by average salary in descending order.**

**Source Code:**

```
SELECT department, COUNT(*) AS employee_count,
ROUND(AVG(salary),2) AS average_salary
FROM employees
GROUP BY department
HAVING COUNT(*) >= 2
ORDER BY average_salary DESC;
```

**Output:**

```
 department | employee_count | average_salary
------------+----------------+----------------
 IT         |              2 |       66000.00
 Finance    |              2 |       52000.00
 Sales      |              3 |       46000.00
(3 rows)
```

## Explanation

**First, I used GROUP BY department to collect the employees of the same department into one group. Then, I used COUNT(*) to count the employees and ROUND(AVG(salary),2) to find the average salary of every department. After that, I used HAVING COUNT(*) >= 2 to remove the departments with fewer than two employees, so HR (only one employee) is not shown in the result. Finally, I used ORDER BY to display the department with the highest average salary first. This task helped me understand the difference between WHERE (filters rows) and HAVING (filters groups).**

# Task 4

**Create the departments table and write SQL queries that:**
**Combine employees and departments using INNER JOIN.**
**Display all employees with their managers using LEFT JOIN.**
**Display all departments with their employees using RIGHT JOIN.**

**Source Code:**

```
CREATE TABLE departments (
department VARCHAR(40) PRIMARY KEY,
manager_name VARCHAR(80) NOT NULL
);

INSERT INTO departments VALUES
('Sales','Amit Mehta'),
('HR','Kavita Rao'),
('IT','Sanjay Jain'),
('Finance','Pooja Nair'),
('Marketing','Vikas Roy');


SELECT e.employee_name,
e.department,d.manager_name
FROM employees e
INNER JOIN departments d
ON e.department=d.department
ORDER BY e.employee_id;

SELECT e.employee_name,
e.department,
d.manager_name
FROM employees e
LEFT JOIN departments d
ON e.department=d.department
ORDER BY e.employee_id;

SELECT e.employee_name,
e.department,
d.manager_name
FROM employees e
RIGHT JOIN departments d
ON e.department=d.department
ORDER BY d.department,e.employee_id;
```

**Output:**

```
 employee_name | department | manager_name
---------------+------------+--------------
 Aarav Sharma  | Sales      | Amit Mehta
 Priya Verma   | HR         | Kavita Rao
 Rahul Singh   | IT         | Sanjay Jain
 Neha Gupta    | Sales      | Amit Mehta
 Kabir Khan    | IT         | Sanjay Jain
 Isha Patel    | Finance    | Pooja Nair
 Rohan Das     | Sales      | Amit Mehta
 Meera Joshi   | Finance    | Pooja Nair
(8 rows)
```

```
 employee_name | department | manager_name
---------------+------------+--------------
 Aarav Sharma  | Sales      | Amit Mehta
 Priya Verma   | HR         | Kavita Rao
 Rahul Singh   | IT         | Sanjay Jain
 Neha Gupta    | Sales      | Amit Mehta
 Kabir Khan    | IT         | Sanjay Jain
 Isha Patel    | Finance    | Pooja Nair
 Rohan Das     | Sales      | Amit Mehta
 Meera Joshi   | Finance    | Pooja Nair
(8 rows)
```

```
 employee_name | department | manager_name
---------------+------------+--------------
 Isha Patel    | Finance    | Pooja Nair
 Meera Joshi   | Finance    | Pooja Nair
 Priya Verma   | HR         | Kavita Rao
 Rahul Singh   | IT         | Sanjay Jain
 Kabir Khan    | IT         | Sanjay Jain
               |            | Vikas Roy
 Aarav Sharma  | Sales      | Amit Mehta
 Neha Gupta    | Sales      | Amit Mehta
 Rohan Das     | Sales      | Amit Mehta
(9 rows)
```

## Explanation

**First, I created the departments table that stores the manager of each department. Then, I joined it with the employees table using the common column department. INNER JOIN displays only the rows that match in both tables. LEFT JOIN displays all employees and their managers; because every employee belongs to a department that exists, the result is the same as the INNER JOIN. RIGHT JOIN displays all departments, so the Marketing department appears with empty (NULL) employee name and department because no employee works in it. This task helped me understand how data from two tables can be combined.**

# Task 5

**Write SQL queries using subqueries to:**
**Display the employees whose salary is greater than the average salary.**
**Create a products table and display the products whose price is greater than the average price.**
**Display the average price of the products.**

**Source Code:**

```
CREATE TABLE products (
product_id VARCHAR(10) PRIMARY KEY,
product_name VARCHAR(50),
category VARCHAR(40),
price NUMERIC(10,2),
stock INT
);

INSERT INTO products VALUES
('P101','Laptop','Electronics',55000,5),
('P102','Mouse','Accessories',700,40),
('P103','Keyboard','Accessories',1500,25),
('P104','Monitor','Electronics',12000,12),
('P105','Chair','Furniture',6500,8),
('P106','Desk','Furniture',9000,4);

SELECT employee_name,salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;

SELECT product_name,price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;

SELECT ROUND(AVG(price),2)
AS average_product_price
FROM products;
```

**Output:**

```
 employee_name |  salary
---------------+----------
 Kabir Khan    | 70000.00
 Rahul Singh   | 62000.00
 Meera Joshi   | 56000.00
 Neha Gupta    | 52000.00
(4 rows)
```

```
 product_name |  price
--------------+----------
 Laptop       | 55000.00
(1 row)
```

```
 average_product_price
-----------------------
              14116.67
(1 row)
```

## Explanation

**First, I wrote a subquery (SELECT AVG(salary) FROM employees) inside the WHERE clause. The inner query finds the average salary (51500) and the outer query displays only the employees earning more than that. Then, I created the products table, inserted six products and used the same idea to display the products whose price is greater than the average price; only the Laptop is above the average. Finally, I displayed the average product price, which is 14116.67. This task helped me understand how one query can be nested inside another query.**

# Task 6

**Use the product data in Excel to:**
**Calculate the stock value of every product (Price × Stock).**
**Use SUM, AVERAGE, MAX, MIN and COUNTA functions on the data.**
**Display the summary of the inventory.**

**Formulas Used:**

```
F2   =D2*E2               (copied down to F7)
I2   =COUNTA(A2:A7)       -> Total Products
I3   =SUM(E2:E7)          -> Total Stock
I4   =AVERAGE(D2:D7)      -> Average Price
I5   =MAX(D2:D7)          -> Highest Price
I6   =MIN(D2:D7)          -> Lowest Price
I7   =SUM(F2:F7)          -> Total Stock Value
```

**Output:**

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | **Product_ID** | **Product_Name** | **Category** | **Price** | **Stock** | **Stock_Value** |
| 2 | P101 | Laptop | Electronics | 55000 | 5 | 275000 |
| 3 | P102 | Mouse | Accessories | 700 | 40 | 28000 |
| 4 | P103 | Keyboard | Accessories | 1500 | 25 | 37500 |
| 5 | P104 | Monitor | Electronics | 12000 | 12 | 144000 |
| 6 | P105 | Chair | Furniture | 6500 | 8 | 52000 |
| 7 | P106 | Desk | Furniture | 9000 | 4 | 36000 |

| | H | I |
|---|---|---|
| 1 | **Summary** | **Result** |
| 2 | Total Products | 6 |
| 3 | Total Stock | 94 |
| 4 | Average Price | 14116.67 |
| 5 | Highest Price | 55000 |
| 6 | Lowest Price | 700 |
| 7 | Total Stock Value | 572500 |

## Explanation

**First, I used the product data containing Product_ID, Product_Name, Category, Price and Stock. Then, I added a new column Stock_Value using the formula =D2*E2 and copied it down to calculate the stock value of every product. After that, I used COUNTA() to count the products, SUM() to find the total stock and total stock value, AVERAGE() to find the average price, MAX() to find the highest price and MIN() to find the lowest price. This task helped me understand how basic functions summarize data in Excel.**

# Task 7

**Use the Excel functions IF(), COUNTIF() and SUMIF() to:**
**Classify each employee salary as High (50000 or more) or Standard.**
**Count the number of sales orders of every employee from the Sales Data sheet.**
**Calculate the total sales of every employee.**

**Formulas Used:**

```
D2   =IF(C2>=50000,"High","Standard")
E2   =COUNTIF('Sales Data'!$C$2:$C$11,A2)
F2   =SUMIF('Sales Data'!$C$2:$C$11,A2,'Sales Data'!$G$2:$G$11)
(all three formulas are copied down to row 9)
```

**Output:**

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | **Employee_Name** | **Department** | **Salary** | **Salary_Status** | **Sales_Orders_Count** | **Sales_Total** |
| 2 | Aarav Sharma | Sales | 45000 | Standard | 2 | 67000 |
| 3 | Priya Verma | HR | 38000 | Standard | 1 | 2100 |
| 4 | Rahul Singh | IT | 62000 | High | 1 | 24000 |
| 5 | Neha Gupta | Sales | 52000 | High | 2 | 9500 |
| 6 | Kabir Khan | IT | 70000 | High | 1 | 55000 |
| 7 | Isha Patel | Finance | 48000 | Standard | 1 | 13000 |
| 8 | Rohan Das | Sales | 41000 | Standard | 1 | 3500 |
| 9 | Meera Joshi | Finance | 56000 | High | 1 | 9000 |

| | A | B | C |
|---|---|---|---|
| 10 | **Function** | **Example** | **Output** |
| 11 | IF() | IF(C2>=20000,"Good","Average") | Good |
| 12 | COUNTIF() | COUNTIF(B2:B6,"IT") | 2 |
| 13 | SUMIF() | SUMIF(B2:B6,"IT",C2:C6) | 132000 |

## Explanation

**First, I used IF() to check the salary of each employee: if the salary is 50000 or more the result is "High", otherwise "Standard". Then, I used COUNTIF() to count how many times every employee name appears in the Customer_Name column of the Sales Data sheet. After that, I used SUMIF() to add the Total_Sales of all the orders that match the employee name. For example, Aarav Sharma has 2 orders with a total of 67000. This task helped me understand conditional functions in Excel.**

# Task 8

**Use the employee data (Employee_ID, Employee_Name, Department, Salary) in Excel to:**
**Find employee details using VLOOKUP().**
**Find employee details using XLOOKUP().**

**Formulas Used:**

```
-- VLOOKUP (Emp_Id entered in I10)
J10  =VLOOKUP(I10,A2:D21,2,FALSE)      -> Emp_Name
K10  =VLOOKUP(J10,B2:E21,2,FALSE)      -> Dept
L10  =VLOOKUP(K10,C2:F21,2,FALSE)      -> Salary

-- XLOOKUP (Emp_Id entered in J13)
J13  E112
J14  =XLOOKUP(J13,$A$2:$A$21,$B$2:$B$21,"Not Found")   -> Emp_Name
J15  =XLOOKUP(J13,$A$2:$A$21,$C$2:$C$21,"Not Found")   -> Dept
J16  =XLOOKUP(J13,$A$2:$A$21,$D$2:$D$21,"Not Found")   -> Salary
```

**Output:**

| | I | J | K | L |
|---|---|---|---|---|
| 7 | VLOOKUP | | | |
| 8 | **Emp_Id** | **Emp_Name** | **Dept** | **Salary** |
| 9 | E104 | Neha | Finance | 40000 |
| 10 | | | | |
| 11 | XLOOKUP | | | |
| 12 | Emp_Id | E112 | | |
| 13 | Emp_Name | Sneha | | |
| 14 | Dept | Sales | | |
| 15 | Salary | 36000 | | |

## Explanation

**First, I used the employee table of 20 employees. In VLOOKUP(), I entered the employee ID E104 and the function searched it in the first column of the table and returned the name Neha, then her department Finance and her salary 40000. Then, I used XLOOKUP() with the employee ID E112; it searched the ID in the Employee_ID column and returned the matching Name, Department and Salary (Sneha, Sales, 36000) from the other columns. I also gave "Not Found" as the message if the ID does not exist. This task helped me understand the difference between VLOOKUP and XLOOKUP.**

# Task 9

**Create a Pivot Table in Excel that:**
**Uses the employee data.**
**Displays the total salary of every department.**

**Steps Used:**

```
Steps:
1. Select the employee data (A1:D21).
2. Insert -> PivotTable -> New Worksheet / Existing Worksheet.
3. Drag  Department  to the  Rows  area.
4. Drag  Salary  to the  Values  area  (Sum of Salary).
```

**Output:**

| | K | L |
|---|---|---|
| 4 | **Row Labels** | **Sum of Salary** |
| 5 | Finance | 162000 |
| 6 | HR | 114500 |
| 7 | IT | 230000 |
| 8 | Marketing | 92000 |
| 9 | Sales | 132000 |
| 10 | **Grand Total** | **730500** |

## Explanation

**First, I selected the employee data and inserted a Pivot Table. Then, I dragged the Department field into the Rows area and the Salary field into the Values area so that Excel calculated the Sum of Salary for every department. The result shows Finance 162000, HR 114500, IT 230000, Marketing 92000 and Sales 132000, and the Grand Total of all salaries is 730500. This task helped me understand how a Pivot Table summarizes a large amount of data quickly.**